Purpose & Audience/Conversation Flow

GEEK Streaming Support Assistant

Project Overview, Audience, and Conversation Design

Current implementation: Landbot proof of concept

Project Purpose

GEEK Streaming Support Assistant is a guided troubleshooting experience for people who need help with common streaming-device problems. It turns a frustrating support search into a clear conversation that identifies the issue, recommends practical first-line checks, and prepares unresolved cases for escalation.

Audience

The primary audience is everyday users who may not feel confident troubleshooting hardware, network, audio/video, or remote-control problems on their own. The experience uses plain language, short prompts, and button-based choices so users can move forward without technical jargon.

Problem Statement

Traditional support pages often give every user the same long list of instructions or make them guess which department to contact. This project narrows the problem through targeted questions and delivers only the guidance that matches the user's situation.

Current Scope

- Four top-level support categories: Power, Connection, Audio/Video, and Remote
- Two symptom paths within each category
- Targeted troubleshooting instructions followed by a resolution check
- Success messages for resolved issues
- A support-request form that collects the user's name, email, device type, issue description, and contact permission when escalation is needed
- Navigation that allows users to request more help or return to the main menu

Current Product Boundary

The Landbot build collects and summarizes support details, but it is not yet connected to an external ticketing platform or live-agent queue. References to ticket creation in the early Python logic and flowchart represent the intended escalation concept. A future version will connect the intake form to a working ticket system.

Detailed Troubleshooting Coverage

Each symptom route uses one diagnostic question to select a troubleshooting action, followed by a final yes/no resolution check. This structure creates four valid end-to-end outcomes for each symptom and 32 total paths across the Python prototype.

Power

- Device will not turn on: If a status light is visible, check the TV input and HDMI connection. If no light is visible, secure the power cable at both the wall outlet and the device. Then verify whether the issue is resolved.
- Device keeps restarting: If the device is powered by the TV's USB port, move it to a wall outlet with the power adapter. If it is already connected to a wall outlet, unplug it for 60 seconds and reconnect it. Then verify whether the restarting stops.

Connection

- No Internet: If Wi-Fi also fails on other devices, restart the router for 30 seconds. If Wi-Fi works elsewhere, forget and reconnect the network on the streaming device. Then verify whether the connection returns.
- Buffering: If the device is hidden in a cabinet or behind a heavy TV, move it to an open area. If placement is already clear, lower the video quality or restart the device. Then verify whether buffering stops.

Audio/Video

- Blank or black screen: If the HDMI cable and port have not been verified, secure the cable or try another port. If they have already been checked, try another HDMI cable and change the TV input. Then verify whether the picture returns.
- No sound: If the TV or soundbar is muted or the volume is below 5, unmute it and raise the volume to 15 for testing. If the volume is already correct, change the audio output to Stereo or Auto. Then verify whether sound returns.

Remote

- Unresponsive remote: If the batteries were not recently replaced, install fresh batteries after pressing each button once to drain leftover power. If the batteries are already new, hold the Home button for 10–15 seconds near the device to force a re-pair. Then verify whether the remote responds.
- Voice features: If the device is not signed in and online, connect it to the internet and sign in. If it is already online, unpair the remote, restart the device, and pair it again. Then verify whether voice control works.

Original Escalation Concepts

In the early Python logic and flowchart, unresolved routes ended in simulated ticket and routing messages. These labels document the intended support destination. They are not active integrations in the current Landbot build.

- Hardware, warranty, and replacement: unresolved power, HDMI/video, or remote-hardware symptoms.
- Network and Tier 2 support: persistent no-internet, buffering, boot-loop, pairing, or software problems.
- App and account support: app failures, authentication issues, and voice or profile settings.
- Current Landbot endpoint: collects the user's details, saves the form fields, and displays a personalized confirmation. External ticket creation and live handoff are planned for a future version.

Conversation Structure

Main Menu

- Power: Device will not turn on or keeps restarting
- Connection: No-internet errors or buffering
- Audio/Video: Blank screen or no sound
- Remote: Unresponsive controls or voice-feature problems

Conversation Flow Architecture

- Entrance and greeting: welcomes the user and explains the assistant's purpose.
- Main-menu selection: routes the user to Power, Connection, Audio/Video, or Remote support.
- Symptom triage: narrows the chosen category to one of two specific problems.
- Diagnostic split: asks one focused question that determines which troubleshooting action appears.
- Targeted troubleshooting: gives a practical first-line step based on the diagnostic answer.
- Resolution verification: asks whether that action fixed the problem.
- Resolved outcome: confirms success and offers a way to continue or return to the main menu.
- Unresolved outcome: moves the user to support-request collection and prepares the case for future escalation.

Design Principles

- Simple: one decision at a time
- Supportive: clear language without blame or unnecessary jargon
- Consistent: repeated interaction patterns across all four categories

- Honest: current features and future integrations are clearly separated
- Expandable: the structure can support new devices, symptoms, integrations, and analytics later

Conversation Flow Chart

Conversation Flow Map

Text Companion to the Saved Flowchart Images

How to Read the Flow

Each route follows the same core pattern: greeting → main menu → symptom selection → diagnostic question → troubleshooting step → resolution check → success or escalation preparation.

Core Branches

Power

- Will not turn on: check the status light, TV input, HDMI connection, and power cable.
- Keeps restarting: confirm the power source, move the device from USB power to a wall adapter when needed, or complete a power cycle.

Connection

- No internet: check whether Wi-Fi works on other devices, restart the router when needed, or forget and reconnect to the network.
- Buffering: check device placement, reduce signal blockage, adjust video quality, or restart the device.

Audio/Video

- Blank screen: secure or replace the HDMI cable, try another port, and verify the TV input.
- No sound: check mute and volume settings, then test a different audio-output format.

Remote

- Unresponsive remote: replace the batteries, drain residual power, or force the remote to re-pair.
- Voice features: confirm the account and internet connection, then unpair and re-pair the remote.

Outcome Logic

- Resolved: display a category-specific success message and offer the user a way to continue or return to the main menu.
- Unresolved: collect support-request details and prepare the case for escalation.
- Invalid or unexpected selection: provide a clear recovery path instead of leaving the user at a dead end.

Path Count and Progression

The Python prototype contains 32 valid end-to-end test paths. The count comes from eight symptom routes, two possible diagnostic answers within each route, and two final outcomes: resolved or unresolved.

The saved flowchart captures the original decision structure. The final Landbot experience adds practical interface features that were not part of the first Python version, including main-menu navigation, a “need more help” loop, an escalation intake form, field validation, and personalized confirmation messages. Those differences are intentional evidence of project growth.

Explanation of Purpose & Logic

GEEK Streaming Support Assistant

Purpose, Logic, Development Progression, and Validation

Developer: Evangela G. | Portfolio Project Documentation

1. Project Purpose and Problem

Streaming-device users often face power failures, network interruptions, blank screens, sound problems, and remote-control issues. Support resources can be text-heavy, overly technical, or disconnected from the user's exact symptom. GEEK Streaming Support Assistant was designed to shorten that path by turning common troubleshooting into a guided conversation.

The assistant uses structured choices to identify the problem, provide a relevant first-line action, and confirm whether the action worked. When the issue remains unresolved, the current Landbot version collects the information needed to prepare the request for escalation.

2. Development Progression

Phase 1: Python Logic Prototype

The project began as a Python decision tree in Google Colab. If/elif/else logic was used to model four categories, eight symptom branches, diagnostic answers, troubleshooting responses, success outcomes, escalation outcomes, and invalid-input handling. Testing identified 32 valid end-to-end paths.

Phase 2: Visual Flowchart

The decision tree was translated into flowchart images so the branches, checkpoints, and endpoints could be reviewed visually. This step made it easier to spot missing connections and confirm that each troubleshooting route reached a clear outcome.

Phase 3: Landbot Implementation

The logic was then rebuilt as an interactive no-code chatbot. Landbot expanded the experience beyond the original prototype with buttons, reusable navigation, branded styling, an escalation form, saved user fields, and a personalized confirmation message.

3. Conversation Logic

- The assistant begins with four issue categories: Power, Connection, Audio/Video, and Remote.
- Each category divides into two symptom-specific paths.
- A diagnostic question determines which troubleshooting instruction the user receives.
- A yes/no resolution check determines whether the interaction ends successfully or moves to escalation preparation.
- Recovery options help users request more help or return to the main menu.

4. Validation Approach

- Python logic testing: use the 32-path spreadsheet to record inputs, expected outcomes, actual outcomes, pass/fail status, and notes.
- Flowchart validation: compare every category, symptom, decision, and endpoint against the Python logic and note intentional differences.
- Landbot visual testing: screen-record representative successful and unresolved paths, verify all buttons and return links, submit the escalation form, confirm saved fields appear correctly, and test required-field validation.
- Cross-version review: document added Landbot features as iteration rather than forcing the earlier Python and flowchart artifacts to match the final interface.

5. Current Capabilities and Boundaries

Implemented

- Guided troubleshooting across four support categories
- Branch-based instructions and resolution checks
- Main-menu and continued-help navigation
- Support-request data collection and personalized confirmation
- Branded Landbot interface

Planned for a Future Version

- Connection to an external ticketing or work-order system
- Automatic transfer to a live support queue
- Natural-language issue classification
- Support analytics and recurring-issue reporting
- Additional device-specific troubleshooting branches

6. Skills Demonstrated

- Conditional logic and decision-tree design
- Conversational UX writing and user-centered troubleshooting
- Python prototyping and structured test planning
- Flowchart mapping and branch validation
- No-code development in Landbot
- Form-field configuration, personalization, and error correction
- Version awareness, scope control, and honest technical documentation

7. Project Goals

- Reduce customer frustration during common streaming device issues
- Route users to the correct solution faster
- Demonstrate conversational flow design and troubleshooting logic
- Showcase no-code development and decision-tree architecture

Personal Project Reflection

Personal Project Reflection

What worked well during this project?

What worked best during this project was slowing down and planning before building. Instead of jumping straight into Landbot or Google Colab, I mapped out every conversation path in my notebook first. Writing the logic by hand helped me visualize how users would move through the troubleshooting process and made it easier to identify gaps before implementation. By the time I started building, I already had a clear roadmap for the chatbot.

What challenges did I face, and how did I work through them?

One challenge I encountered was realizing that users do not always follow the expected path. My original design focused heavily on troubleshooting steps but did not fully account for invalid selections, failed resolutions, or situations where a user might need additional guidance. As I built and tested the Landbot version, those gaps showed up in specific ways, including return-to-menu links, agent selection, field setup, checkbox values, and form connections. I slowed down, traced each connection, renamed similar items, and tested the flow again. That process made the chatbot more realistic and improved the overall user experience.

How did the project change from the original plan?

The Python version focused on the core troubleshooting logic. The flowchart made that logic visual. Once I built the Landbot version, I added features that made the experience feel more complete, including a “need more help” loop, main-menu navigation, an escalation form, required fields, saved answers, personalization, and a branded design. I am keeping those differences because they show real progression instead of pretending the first draft and final prototype were identical.

What did I learn from testing?

I learned that every button, connector, saved field, and endpoint has to be tested from the user’s point of view. The Python logic has 32 valid end-to-end paths, which made it clear that a project can look simple on the main menu while still containing many possible journeys underneath. A test record gives me proof that the logic works and helps me separate confirmed behavior from assumptions.

What would the next version improve?

The next version would connect the escalation form to a real ticketing or work-order system so the collected information has a destination after submission. It could also include a live-agent handoff, natural-language issue detection, support analytics, and more device-specific

troubleshooting. Those features would move the project from a polished proof of concept toward a production-ready support tool.

Final Reflection

This project reminded me that I enjoy the part where logic, design, and problem-solving meet. I did not just build a chatbot that moves through questions. I learned how to plan a support experience, test it, find what was missing, and improve it. The finished Landbot reflects my style: earthy, simple, clear, and helpful without feeling cold or overly technical.